Ouster BlueCity Traffic Data API Manual

Version: 1.0 | **Date:** August 31, 2026 | **Maintainer:** Ouster BlueCity

Table of Contents

Ouster BlueCity Traffic Data API Manual 1
Table of Contents 1
1. Introduction 4
1.1 About This Manual 4
1.2 API Categories at a Glance 4
Choosing the Right API 4
1.3 Document Conventions 4
1.4 Revision History 5
2. Common Reference 6
2.1 Authentication 6
2.1.1 Scheme A — JSON Web Token (JWT) via core.api.bluecity.ai 6
2.1.2 Scheme B — Edge Client Credentials 7
2.2 Identifiers and Time Conventions 7
2.3 Movement Codes 8
2.4 Object Class Codes 9
2.5 Signal State Values 9
2.6 Errors and Status Codes 9
2.7 Upstream Data Source — Ouster Gemini Detect 10
3. Category 1 — Cloud-Based API 11
3.1 Overview 11
3.2 Core APIs 11
3.2.1 Turning Movement Counts (TMC) 11
3.3 Safety Analysis APIs 13
3.3.1 Post Encroachment Time (PET) 13
3.3.2 Average Speed 14
3.4 Signal Performance Measures (SPM) APIs 16
3.4.1 Phase Details — Phase Discovery 16
3.4.2 Jaywalking — Disaggregated Events 17
3.4.3 Occupancy Ratio — Per-Cycle Ratios 18
3.4.4 Occupancy Ratio — Averages by Signal State 19

3.4.5 Occupancy Ratio — Split Trend Daily Percentages 20
3.4.6 Occupancy Ratio — Split Trends (GoR/RoR5) 21
3.4.7 Phase Interval — Arrivals 22
3.4.8 Phase Interval — Green Allocation 22
3.4.9 Phase Interval — Simple Approach Delay 23
3.4.10 Phase Interval — Phase Interval Raw Data 24
3.4.11 Red-Light Runner — Daily Aggregates 25
3.4.12 Red-Light Runner — Disaggregated Events 26
4. Category 2 — Edge-Based API 28
4.1 Overview 28
4.2 Authentication 28
4.3 Endpoints 29
4.3.1 Raw Zone Object Events (object_events) 29
4.3.2 Turning Movements (turning_movements) 31
4.3.3 Zone Metadata Aggregation (zone_metadata) 34
4.3.4 Zone Occupancy Ratio Aggregation (zone_occupancy) 36
4.3.5 Zone Queue Length Aggregation (zone_queue_length) 37
5. Category 3 — Real-Time gRPC Streaming API 40
5.1 Overview 40
5.2 Authentication and Access 40
5.3 Communication Flow 40
5.3.1 Routing (Beacon) 41
5.3.2 Connecting (gRPC) 41
5.3.3 Reconnection and Queued Data 42
5.4 Data Structures 42
5.4.1 HyperParameter 42
5.4.2 Frame and FrameObject 43
Real-Time Class Types 44
5.4.3 PhaseChange 45
5.4.4 OccupancyChange 45
5.5 Python Client Module (BCT Realtime Subscriber SDK) 46
5.5.1 Installation 46
5.5.2 Realtime_subscriber_api (username/password) 46
5.5.3 Realtime_subscriber_service (service token) 47
5.5.4 Consuming Data 47
5.5.5 Realtime_subscriber_archive (offline testing) 48
5.6 Building a Custom Client 48
6. Appendices 49
Appendix A — Swagger and Reference Links 49

Appendix B — Glossary 49
Appendix C — Support 50

1. Introduction

1.1 About This Manual

Ouster BlueCity provides lidar-based traffic monitoring at intersections and roadways. The platform exposes traffic, safety, and signal-performance data through three complementary API categories. This manual describes each category, its authentication scheme, its endpoints and data structures, and provides integration examples.

1.2 API Categories at a Glance

Category	Protocol	Data Nature	Authentication
1. Cloud-Based API	REST / HTTPS (JSON)	Historical and aggregated analytics (turning movement counts, post-encroachment time, speed, signal performance metrics)	JWT via core.api.bluecity.ai account (Scheme A)
2. Edge-Based API	REST / HTTPS (JSON), served from the edge box	Zone analytics, turning movement counts computed on the edge device (more in future)	Client name + secret retrieved from the edge box (Scheme B)
3. Real-Time gRPC Streaming API	gRPC / Protocol Buffers over HTTP/2 (TLS)	Live per-frame object data, signal phase changes, virtual-loop occupancy changes	JWT via core.api.bluecity.ai account, or service token (Scheme A)

Choosing the Right API

Use **Category 1** when you need historical or aggregated results over a time range (counts, conflicts, speeds, signal performance measures). Use **Category 2** when your application runs on the same network as the edge device and consumes zone-based analytics directly from it. Use **Category 3** when you need a continuous, low-latency stream of raw detections, traffic-light phase changes, and loop occupancy events as they happen.

Ouster, Inc 2026 - Proprietary & Confidential

1.3 Document Conventions

All timestamps follow the **ISO 8601** standard unless stated otherwise. Endpoints marked **[BETA]** are under active development and may change without notice. Placeholder values in examples are shown in angle brackets, e.g. <YOUR_ACCESS_TOKEN>.

1.4 Revision History

Version	Date	Changes
1.0 (Draft)	2026-08-10	Initial consolidated manual. Adds SPM APIs (beta), Edge-Based Zone Analytics placeholder, and Real-Time gRPC Streaming API. Absorbs content from the BlueCity Analytics API Integration Guide.

2. Common Reference

2.1 Authentication

BlueCity APIs use two authentication schemes. Each API category in this manual states which scheme applies.

2.1.1 Scheme A — JSON Web Token (JWT) via core.api.bluecity.ai

Applies to: Category 1 (Cloud-Based Analytics) and Category 3 (Real-Time gRPC Streaming).
You must hold a user account on **core.api.bluecity.ai**. All requests are authorized with a JWT access token obtained from the token endpoint.

Obtaining a Token

Detail	Value
Endpoint	https://core.api.bluecity.ai/api/token/
Method	POST
Content-Type	application/json
Swagger Reference	https://core.api.bluecity.ai/schema/swagger-ui/#/api/api_token_create

Request body:

```
JSON
{
  "username": "YOUR_USERNAME",
  "password": "YOUR_PASSWORD"
}
```

Successful response (HTTP 200):

```
JSON
{
  "access": "eyJ0eXAiOiJK...",
  "refresh": "eyJ0eXAiOiJK..."
}
```

Using the Token
Include the access token in the header of every subsequent API call:

```
None
Authorization: Bearer <YOUR_ACCESS_TOKEN>
```

Token Expiry and Renewal
Tokens have a short lifespan, expiring after a few days. The exact expiration time can be determined by base64-decoding the token body. You are responsible for renewing tokens. The recommended method is the refresh endpoint:

Detail	Value
Swagger reference	https://core.api.bluecity.ai/schema/swagger-ui/#/api/api_token_refresh_create
Method	POST (send your refresh token in the request body)

Requesting a new token pair with your credentials is also acceptable.

2.1.2 Scheme B — Edge Client Credentials

Applies to: Category 2 (Edge-Based Zone Analytics).
Edge-based endpoints are authenticated with a **client name** and a **client secret** issued through Keycloak, the identity provider embedded in the Ouster Detect GUI. See Section 4.2 for the retrieval procedure.

2.2 Identifiers and Time Conventions

Concept	Description
UDID	The unique identifier of an edge box / sensor (e.g. obc-ag-sa-b123456).
fdate / tdate	Start and end of the query window, in ISO 8601 (e.g. 2026-08-10T00:00:00). Maximum window length varies per endpoint and is stated in each endpoint section.
timezone	Optional IANA timezone name (e.g. US/Eastern, US/Pacific, Europe/Berlin). Default is UTC. When provided, endpoints

Concept	Description
	return additional *_local timestamp fields.
Aggregation levels	Where supported: **1** = 15 minutes, **2** = hourly, **3** = daily, **4** = monthly.

2.3 Movement Codes

Analytics endpoints use standard codes to describe traffic movements at an intersection. Refer to this table when filtering by turning movement or interpreting results.

Code	Direction	Type	Description
NS	North to South	Motorized	Southbound through
SN	South to North	Motorized	Northbound through
EW	East to West	Motorized	Westbound through
WE	West to East	Motorized	Eastbound through
NE	North to East	Motorized	Southbound right turn
NW	North to West	Motorized	Southbound left turn
SE	South to East	Motorized	Northbound right turn
SW	South to West	Motorized	Northbound left turn
EN	East to North	Motorized	Westbound right turn
ES	East to South	Motorized	Westbound left turn
WN	West to North	Motorized	Eastbound left turn
WS	West to South	Motorized	Eastbound right turn
<x>LR	Leg (N/S/E/W)	Crosswalk	Crosswalk Left to Right when facing the intersection (e.g. nlr, slr)
<x>RL	Leg (N/S/E/W)	Crosswalk	Crosswalk Right to Left when facing the intersection (e.g. nrl,

Code	Direction	Type	Description
			srl)

2.4 Object Class Codes

The system uses numerical codes to identify object classes. The following table applies to the Cloud-Based Analytics Data API (Category 1). The Real-Time gRPC API uses an extended variant of this table, documented in Section 5.4.

Code	Description
2	Vehicle
3	Van
4	Truck
5	Bus
6	SUV
8	Trailer
9	Tractor
10	Pedestrian
13	Bicycle
15	Motorcycle
18	Tram

Note: some of these categories are currently included as placeholders and are not actively classified within our integration.

2.5 Signal State Values

SPM endpoints (Section 3.4) report the signal state as a string: green, yellow, red, or redFirst5 (the first 5 seconds of red). The Real-Time gRPC API (Section 5) reports phase status as an integer: 0 = Invalid, 1 = Green, 2 = Yellow, 3 = Red.

2.6 Errors and Status Codes

Status	Meaning
200	Success.
400	Bad request — invalid or missing parameters. The response body contains details.
403	Forbidden — missing, invalid, or expired token, or insufficient access rights for the requested sensor/endpoint.

2.7 Upstream Data Source — Ouster Gemini Detect

Every analytic in this manual originates with **Ouster Gemini Detect**, the perception software running on the edge box. Detect fuses point clouds from one or more lidar sensors into tracked objects — each carrying position, velocity, orientation, bounding-box dimensions, and classification — evaluates those objects against user-defined **event zones**, and publishes the results as JSON data streams through its **LidarHub** component. BlueCity consumes this output and derives the turning movement counts, safety measures, signal performance measures, and zone analytics documented in Sections 3 through 5.

This section is orientation only. Complete field definitions for every stream are maintained in the Ouster Detect documentation:
https://docs.ouster.com/ouster-detect/connecting_to_output/connecting-to-output.html

3. Category 1 — Cloud-Based API

3.1 Overview

The Cloud-Based Analytics Data API provides historical and aggregated traffic and safety analytics over HTTPS. It comprises three API families, each with its own base URL and Swagger page. **All requests use Authentication Scheme A** (JWT from a core.api.bluecity.ai account, Section 2.1.1).

API Family	Primary Data Focus	Base URL	Swagger Reference
Core APIs	Authentication, Turning Movement Counts (TMC)	https://core.api.bluecity.ai	https://core.api.bluecity.ai/schema/swagger-ui/
Safety Analysis APIs	PET, Average Speed	https://safety.api.bluecity.ai	https://safety.api.bluecity.ai/api/schema/swagger-ui/
Signal Performance (SPM) APIs [BETA]	Signal performance measures: occupancy ratio, arrivals, green allocation, delay, red-light runners, jaywalking	https://spm.api.bluecity.ai	https://spm.api.bluecity.ai/api/schema/swagger-ui/

We recommend reviewing the relevant Swagger documentation for the most up-to-date and specific endpoint information.

3.2 Core APIs

3.2.1 Turning Movement Counts (TMC)

Retrieves aggregated traffic counts for a specific sensor. Data can be retrieved as a total sum or categorized by object class (Car, Truck, Bus, Bicycle, Pedestrian, etc.).

Detail	Value
Endpoint	https://core.api.bluecity.ai/api/v2/turningmovement/
Method	GET

Detail	Value
Swagger Reference	https://core.api.bluecity.ai/schema/swagger-ui/#/turningmovement/turningmovement_retrieve

Parameters

Parameter	Type	Required	Description
UDID	String	**Yes**	The unique ID of the edge box / sensor.
fdate	Date-Time	**Yes**	Start time (ISO 8601), e.g. 2023-01-01T14:10:00.
tdate	Date-Time	**Yes**	End time (maximum range: 24 hours).
a	String	No	Aggregation level: 1 = 15 minutes, 2 = hourly, 3 = daily, 4 = monthly.
f	String	No	Fields: comma-separated list of movements to include (e.g. ns,se,ew). Defaults to all.
simplified	Boolean	No	Default: true. If true, returns total counts (all classes summed). If false, returns counts categorized by class.

Sample Response (simplified = false)

```
JSON
{
```

```
  "UDID": "12345",
  "aggregation": 3,
  "values": {
    "2021-01-10 00:00:00-07:00": {
      "Passenger Vehicle": { "ne": 324, "ns": 976 },
      "Pedestrian": { "nlr": 204 }
    }
  }
}
```


3.3 Safety Analysis APIs

3.3.1 Post Encroachment Time (PET)

Returns safety conflict data. Each PET event involves two participants: an **Object** (leading entity) and a **Shadow** (following entity).

Detail	Value
Endpoint	https://safety.api.bluecity.ai/api/analytics/PET
Method	GET
Swagger Reference	https://safety.api.bluecity.ai/api/schema/swagger-ui/

Parameters

Parameter	Type	Required	Description
UDID	String	**Yes**	The unique ID of the edge box.
fdate	Date-Time	**Yes**	Start time (ISO 8601).
tdate	Date-Time	**Yes**	End time (maximum range: 24 hours).
PETMargin	Integer	No	Threshold for PET values. Default is 10.

Parameter	Type	Required	Description
timezone	String	No	Desired timezone for the timestamp (e.g. US/Eastern).
turningMovement	String	No	Filter by specific movements (comma-separated), e.g. NS,SE.

Response Field Definitions

objectID / shadowID: unique identifiers of the entities involved. PETValue: the calculated Post Encroachment Time in seconds. speed / shadowSpeed: speed of the respective entities during the conflict. lat / long: GPS coordinates of the conflict.

Sample Response

JSON


```
[
  {
    "id": 123,
    "objectID": "1234",
    "objectType": 2,
    "shadowID": "53",
    "shadowType": 10,
    "timestamp": "2022-06-01T18:28:25.990Z",
    "timestamp_local": "2022-06-01T20:28:25.990Z",
    "turningMovement": "NS",
    "shadowTurningMovement": "SE",
    "PETValue": 2,
    "speed": 123,
    "positionX": 123,
    "positionY": 123
  }
]
```


3.3.2 Average Speed

Returns the average speed of vehicles per turning movement over a specific time range.

Detail	Value
Endpoint	https://safety.api.bluecity.ai/api/analytics/avgspeed
Method	GET
Swagger Reference	https://safety.api.bluecity.ai/api/schema/swagger-ui/

Parameters

Parameter	Type	Required	Description
UDID	String	**Yes**	The unique ID of the edge box.
fdate	Date-Time	**Yes**	Start time (ISO 8601).
tdate	Date-Time	**Yes**	End time (maximum range: 24 hours).
aggregation	Integer	No	Aggregation level: 1 = 15 minutes, 2 = hourly, 3 = daily, 4 = monthly.
timezone	String	No	Desired timezone (e.g. US/Eastern).
turningMovement	String	No	Filter specific movements (e.g. NS,SE).

Sample Response

JSON

```
{
  "UDID": "<UDID>",
  "date_from": "<DATE_FROM>",
  "date_to": "<DATE_TO>",
  "aggregation": "<AGGREGATION>",
  "fields": ["NS", "SN", "WE", "EW"],
  "values": {
    "<DATETIME1>": {
```

```
    "EW": "<speed in km/h>",
    "NS": "<speed in km/h>",
    "SN": "<speed in km/h>",
    "WE": "<speed in km/h>"
   }
  }
}
```

3.4 Signal Performance Measures (SPM) APIs

Status: under development. These endpoints are served from the development environment at **https://spm.api.bluecity.ai** and are subject to change without notice. All endpoints use the GET method and Authentication Scheme A (JWT Bearer). Unless stated otherwise, fdate/tdate are required ISO 8601 timestamps and timezone is optional (default UTC).
Two identifier styles are used: sensor-scoped endpoints take a **UDID**; phase-scoped endpoints take a **phase_id** (or a comma-separated **phase_ids** list) referring to a traffic signal phase configured for the intersection.

3.4.1 Phase Details — Phase Discovery

Lists the phases configured on a sensor. Use this endpoint to discover which phase IDs belong to a sensor before querying the phase-scoped endpoints (occupancy ratio, green allocation, approach delay, arrivals, split trends). Each record describes one phase: its ID, a human-readable label, the approach leg it serves, whether it is currently active, and the movements it allows.

Phases are versioned. When a phase's configuration changes, the existing phase is marked inactive and a new phase — with a new phase_id — is created and marked active in its place. A sensor can therefore carry several phases that represent the same physical movement at different points in time. Pass active_only=true to return only the phases in effect now, or active_only=false to also include historical (inactive) phases.

Detail	Value
Endpoint	https://spm.api.bluecity.ai/api/v2/spm/phase/details/customer/
Method	GET

Parameters

Parameter	Type	Required	Description
UDID	String	**Yes**	Sensor UDID
active_only	Boolean	No	true returns only phases currently in effect: false also returns historical (inactive) phases.

Sample Response

```
JSON
[
  {
    "phase_id": 42,
    "phase_label": "NB Through",
    "approach": "N",
    "active": true,
    "allowed_movements": "ns,ne",
    "phase_type": 1

  }
]
```

3.4.2 Jaywalking — Disaggregated Events

Returns disaggregated jaywalking events within the selected time window. One record per event, ordered newest-first. Limited to 1000 records per request.

Detail	Value
Endpoint	https://spm.api.bluecity.ai/api/v2/spm/jayWalking/events/
Method	GET

Parameters

Parameter	Type	Required	Description
UDID	String	**Yes**	Sensor UDID (e.g. obc-ag-sa-b123456) .
fdate / tdate	Date-Time	**Yes**	Query window (ISO 8601).
timezone	String	No	IANA timezone; default UTC.

Key Response Fields
timestamp_enter/timestamp_exit (UTC, plus *_local variants), time_difference (seconds spent in the monitored zone), sensor, object_id, class_type, trajectory (flat [x, y, x, y, ...] array in metres, sensor-local frame), entry_point ([x, y]), distance_to_sensor (metres), recording_path and recording_detect_version (null if no video clip was captured).

Sample Response

JSON


```
[
  {
    "timestamp_enter": "2024-03-01T13:04:22.183000Z",
    "timestamp_enter_local": "2024-03-01T08:04:22.183000",
    "timestamp_exit": "2024-03-01T13:04:35.901000Z",
    "timestamp_exit_local": "2024-03-01T08:04:35.901000",
    "time_difference": 13.718,
    "sensor": "ABC123XYZ",
    "class_type": 3,
    "trajectory": [1.2, 0.5, 1.8, 1.1, 2.4, 1.9, 3.0, 2.7]

  }
]
```


3.4.3 Occupancy Ratio — Per-Cycle Ratios

Returns per-cycle occupancy ratios for a phase over the requested time window. Each record corresponds to one signal cycle, identified by its phase start and end timestamps. ratio is expressed as a percentage (0–100). Maximum time window: 31 days.

Detail	Value
Endpoint	https://spm.api.bluecity.ai/api/v2/spm/occupancyRatio/ratios/customer/
Method	GET

Parameters

Parameter	Type	Required	Description
phase_id	String	**Yes**	Signal phase identifier (e.g. 42).
fdate / tdate	Date-Time	**Yes**	Query window (ISO 8601), max 3 days.
traffic_light_statuses	String	No	Comma-separated signal states to include: green, yellow, red, redFirst5.
timezone	String	No	IANA timezone; default UTC.

Sample Response

JSON


```
[
  {
    "phase_start": "2024-03-01T13:04:00Z",
    "phase_start_local": "2024-03-01T08:04:00",
    "phase_end": "2024-03-01T13:05:05Z",
    "phase_end_local": "2024-03-01T08:05:05",
    "phase": 42,
    "ratio": 67.3,
    "traffic_light_status": "green",
    "phase_duration": 65.0
  }
]
```


3.4.4 Occupancy Ratio — Averages by Signal State

Returns the mean occupancy ratio per signal state for a phase over the requested time

window. Produces one record per traffic_light_status value present in the data (up to four: green, yellow, red, redFirst5). Maximum time window: 31 days.

Detail	Value
Endpoint	https://spm.api.bluecity.ai/api/v2/spm/occupancyRatio/average/
Method	GET

Parameters

Same as Section 3.4.2 (phase_id, fdate, tdate, optional traffic_light_statuses, timezone).

Sample Response

```
JSON
[
  { "phase": 42, "traffic_light_status": "green", "average_ratio": 61.4 },
  { "phase": 42, "traffic_light_status": "yellow", "average_ratio": 8.2 },
  { "phase": 42, "traffic_light_status": "red", "average_ratio": 34.7 },
  { "phase": 42, "traffic_light_status": "redFirst5", "average_ratio": 41.0 }
]
```

3.4.5 Occupancy Ratio — Split Trend Daily Percentages

Returns split-trend classification percentages for a phase, aggregated into 15-minute bins. Each bin classifies cycles into five categories: **split failure**, **heavy traffic**, **OK**, **random arrivals**, and **spare capacity**. Bins with no classifiable cycles are omitted. Maximum query span: 31 days.

Detail	Value
Endpoint	https://spm.api.bluecity.ai/api/v2/spm/occupancyRatio/splitTrendsDailyPercentages/customer/
Method	GET

Parameters
phase_id, fdate, tdate (required); timezone (optional).

Sample Response

```
JSON
[
  {
    "interval_start": "2024-03-01T17:00:00",
    "interval_end": "2024-03-01T17:15:00",
    "split_failure_percentage": 18.75,
    "heavy_traffic_percentage": 25.0,
    "ok_percentage": 37.5,
    "random_arrivals_percentage": 12.5,
    "spare_capacity_percentage": 6.25,
    "phase_id": 42
  }
]
```

3.4.6 Occupancy Ratio — Split Trends (GoR/RoR5)

Returns the green occupancy ratio (gor) and the red occupancy ratio over the first five seconds of red (ror5) for each signal cycle in a single phase. These are the two raw inputs to split-failure classification; this endpoint does not assign a split-trend category.

Detail	Value
Endpoint	https://spm.api.bluecity.ai/api/v2/spm/occupancyRatio/splitTrends/
Method	GET

Parameters
phase_id, fdate, tdate (required); timezone (optional).

Sample Response

```
JSON
[
  {
    "cycle_start": "2024-03-01T17:00:00Z",
    "cycle_end": "2024-03-01T17:01:30Z",
    "gor": 9.51,
    "ror5": 0.0,
    "phase_id": 42
  }
]
```

3.4.7 Phase Interval — Arrivals

Returns arrival counts for a phase, aggregated into 15-minute bins and broken down by signal state. Each record covers one bin and signal-state combination; arrivals_count is the total number of vehicle arrivals for that state within the bin. interval_start reflects local time when timezone is provided, otherwise UTC. Bins with no arrivals for a given state are omitted.

Detail	Value
Endpoint	https://spm.api.bluecity.ai/api/v2/spm/phaseInterval/arrivals/customer/
Method	GET

Parameters
phase_id, fdate, tdate (required); timezone (optional).

Sample Response

JSON


```
[
  { "interval_start": "2024-03-01T13:00:00Z", "traffic_light_status": "green",
"arrivals_count": 47, "phase_id": 42 },
  { "interval_start": "2024-03-01T13:00:00Z", "traffic_light_status": "red", "arrivals_count":
12, "phase_id": 42 }
]
```


3.4.8 Phase Interval — Green Allocation

Returns green time as a percentage of total cycle time, aggregated into 15-minute bins, for one or more phases. green_allocation = green / (green + yellow + red) across all cycles whose start falls in the bin (0–100). Bins with no cycles are included with a value of 0.0.

Detail	Value
Endpoint	https://spm.api.bluecity.ai/api/v2/spm/phaseInterval/greenAllocation/customer/

Detail	Value
Method	GET

Parameters
phase_ids (comma-separated, e.g. 42,43), fdate, tdate (required); timezone (optional).

Sample Response

JSON


```
[
  { "interval_start": "2024-03-01T13:00:00Z", "interval_end": "2024-03-01T13:15:00Z",
"green_allocation": 42.3, "phase_id": 42 },
  { "interval_start": "2024-03-01T13:00:00Z", "interval_end": "2024-03-01T13:15:00Z",
"green_allocation": 35.8, "phase_id": 43 }
]
```


3.4.9 Phase Interval — Simple Approach Delay

Returns per-cycle approach delay for red phase intervals across one or more phases. One record per red cycle, ordered by phase_start. delay is in seconds. Only red cycles are returned.

Detail	Value
Endpoint	https://spm.api.bluecity.ai/api/v2/spm/phaseInterval/simpleDelay/
Method	GET

Parameters
phase_ids (comma-separated), fdate, tdate (required); timezone (optional).

Sample Response

JSON


```
[
  {
    "phase_start": "2024-03-01T18:15:00Z",
```

```
  "phase_end": "2024-03-01T18:15:47Z",
  "phase_start_local": "2024-03-01T13:15:00",
  "phase_end_local": "2024-03-01T13:15:47",
  "traffic_light_status": "red",
  "delay": 28.4,
  "phase_id": 42
 }
]
```


3.4.10 Phase Interval — Phase Interval Raw Data

Returns raw, per-interval records for one or more phases: one row per signal phase interval with its start and end, signal state (green, yellow, or red), delay in seconds, and arrivals. This is unaggregated data and can be high volume, so the window is capped at 6 hours.

arrivals is an array in which each element is the time, in seconds, at which a vehicle approached the intersection during that interval — one element per arriving vehicle. Both delay and arrivals are null when the value is not available for the interval; in practice delay is populated for red intervals and null for green and yellow.

Detail	Value
Endpoint	https://spm.api.bluecity.ai/api/v2/spm/phaseInterval/phaseIntervalRawData/customer/
Method	GET

Parameters
phase_ids (comma-separated), fdate, tdate (required); timezone (optional).

Sample Response

JSON
```
[
 {
  "phase_start": "2023-10-20T00:00:05.489716Z",
  "phase_end": "2023-10-20T00:01:10.889575Z",
  "traffic_light_status": "red",
  "phase_id": 271,
  "delay": 0,
  "arrivals": [2, 4, 5, 12, 14, 16, 20, 33, 44, 51, 57, 61, 63, 65]
 },
```

```
  {
    "phase_start": "2023-10-20T00:01:10.889575Z",
    "phase_end": "2023-10-20T00:01:42.089890Z",
    "traffic_light_status": "green",
    "phase_id": 271,
    "delay": null,
    "arrivals": [0, 2, 3, 5, 8, 10, 12, 16, 20, 24, 27, 30]
  },
  {
    "phase_start": "2023-10-20T00:55:02.791420Z",
    "phase_end": "2023-10-20T00:55:06.188340Z",
    "traffic_light_status": "yellow",
    "phase_id": 271,
    "delay": null,
    "arrivals": null
  }
]
```


3.4.11 Red-Light Runner — Daily Aggregates

Returns daily red-light-runner event counts for a specific approach leg of a sensor. One record per calendar day within the requested window; interval_start is midnight of that day. rlr_count covers all phases assigned to the specified approach. Days with zero events are omitted.

Detail	Value
Endpoint	https://spm.api.bluecity.ai/api/v2/spm/redLightRunner/aggregated/customer/
Method	GET

Parameters

Parameter	Type	Required	Description
UDID	String	**Yes**	Sensor UDID.
approach	String	**Yes**	Approach leg: N, S, E, or W.
fdate / tdate	Date-Time	**Yes**	Query window (ISO

Parameter	Type	Required	Description
			8601).
timezone	String	No	IANA timezone; default UTC.

Sample Response

```
JSON
[
  { "interval_start": "2024-03-01T00:00:00Z", "rlr_count": 14 },
  { "interval_start": "2024-03-02T00:00:00Z", "rlr_count": 9 },
  { "interval_start": "2024-03-03T00:00:00Z", "rlr_count": 21 }
]
```

3.4.12 Red-Light Runner — Disaggregated Events

Returns individual red-light-runner events detected by a sensor within the requested time window. Each record is one vehicle crossing flagged as a red-light runner.

Detail	Value
Endpoint	https://spm.api.bluecity.ai/api/v2/spm/redLightRunner/events/
Method	GET

Parameters

Parameter	Type	Required	Description
UDID	String	**Yes**	Sensor UDID.
fdate / tdate	Date-Time	**Yes**	Query window (ISO 8601).
timezone	String	No	IANA timezone; default UTC.

Sample Response

JSON


```
[
  {
    "red_light_runner_id": 18423,
    "sensor": "obc-ag-sa-b123456",
    "phase": 7,
    "object_id": 3041872,
    "class_type": 1,
    "timestamp_enter": "2024-03-01T14:22:03.412000Z",
    "timestamp_exit": "2024-03-01T14:22:05.841000Z",
    "speed": 48.3,
    "turning_movement": "ns",
    "phase_start": "2024-03-01T14:22:00.000000Z",
    "traffic_light_status": "red",
    "time_enter_diff": 3.41,
    "time_exit_diff": 5.84,
    "recording_path": null,
    "recording_detect_version": null
  }
]
```

4. Category 2 — Edge-Based API

4.1 Overview

The Edge-Based Zone Analytics API is served directly from the BlueCity edge box on the local network and provides zone-based analytics and turning movements computed on the device. Because the API is hosted on the edge device, its address depends on your deployment (for example: https://<edge-box-address>/analytics/api/v1/, with Swagger documentation at https://<edge-box-address>/analytics/api/v1/swagger/ui/). Or login to the GUI page, then go do Diagnostics -> Service Reachability (bottom right corner) -> expand Analytics Server -> API page.

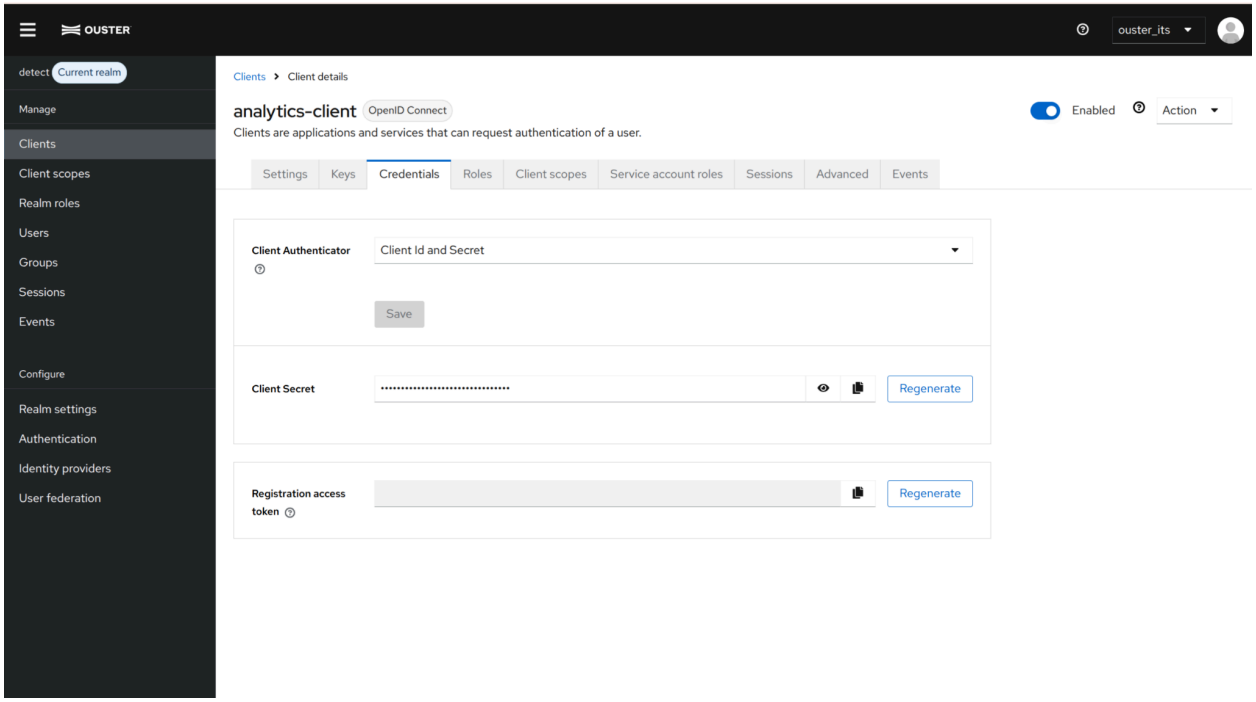


4.2 Authentication

This category uses **Authentication Scheme B**: requests are authenticated with a **client name** and a **client secret** issued by Keycloak, the identity provider embedded in the edge box's Ouster Detect GUI.

To retrieve your credentials:

1. Log in to the Ouster Detect GUI for your edge box.
2. Navigate to **Client Settings** (the embedded Keycloak administration page).
3. Locate the client entry for the integration you are building — or create one, if your role permits — and note its **Client ID** (this is the client name).
4. Retrieve the corresponding **Client Secret** from the same page, regenerating it if necessary.



4.3 Endpoints

This section documents five endpoints of the Analytics Server API: raw zone object events, three aggregated zone-analytics endpoints (zone metadata, zone occupancy, and zone queue length), and a turning movement endpoint. Other endpoints in the spec — server metadata, application configuration, server reset, SNMP zone mappings, and the corresponding /batch ingest endpoints — can be referred to in the swagger page. Unless stated otherwise, endpoints use Authentication Scheme B (Section 2.1.2 / 4.2) and are hosted at https://<edge-box-address>/analytics/api/v1/.

4.3.1 Raw Zone Object Events (object_events)

Retrieves individual zone metadata records (object events) without aggregation, within a specified time range. Results are ordered chronologically by timestamp, ascending, and all measurements are converted to the configured unit system.

Detail	Value
Endpoint	https://<edge-box-address>/analytics/api/v1/object_events
Method	GET

Parameters

Parameter	Type	Required	Description
start_time	Date-Time	No	Start of the query window (ISO 8601). Defaults to 5 minutes before end_time.
end_time	Date-Time	No	End of the query window (ISO 8601). Defaults to the current time.
timezone	String	No	Overrides the configured timezone for returned timestamps.
imperial_measuring_unit	Boolean	No	Overrides the configured unit system.
deduplicate_objects	Boolean	No	Overrides the configured deduplicate_objects setting.
page	Integer	No	Page number to return, 1-based. Defaults to 1.
per_page	Integer	No	Records per page. Defaults to 1000; hard maximum of 5000.

Sample Response

JSON


```
{
  "query_start_time": "2026-08-10T11:55:00Z",
  "query_end_time": "2026-08-10T12:00:00Z",
  "interval_seconds": 300,
  "total_buckets": 1,
  "timezone": "UTC",
  "units": "metric",
```

```
"buckets": [
  {
    "bucket_start": "2026-08-10T11:55:00Z",
    "bucket_end": "2026-08-10T12:00:00Z",
    "id": 48213,
    "perception_id": 3,
    "timestamp": "2026-08-10T11:58:42.183Z",
    "created_at": "2026-08-10T11:58:42.500Z",
    "object_id": "3f29b1de-8f2f-4b7a-9e2c-6a1a2c3d4e5f",
    "zone_name": "Zone-14",
    "zone_id": 14,
    "dwell_time_ms": 3200,
    "num_samples": 42,
    "avg_speed": 24.6,
    "p50_speed": 23.9,
    "p85_speed": 29.1,
    "avg_height": 1.6,
    "avg_length": 4.5,
    "avg_width": 1.8,
    "classification": "VEHICLE",
    "sub_classification": "car",
    "user_classification": null,
    "speed_bin": "Moderate"
  }
 ]
}
```


4.3.2 Turning Movements (turning_movements)

Retrieves completed turning movements aggregated into fixed-length time buckets, grouped by entry/exit zone pair. Each bucket reports the number of objects that completed a given movement within the interval, along with classification, sub-classification, and user-classification counts for that movement. Movements are derived on the edge box from the ordered sequence of zones each object traverses; only the entry (first) and exit (last) zone of that path are exposed by this endpoint.

Detail	Value
Endpoint	https://<edge-box-address>/analytics/api/v1/turning_movements
Method	GET

Parameters

Parameter	Type	Required	Description
start_time	Date-Time	No	Start of the query window (ISO 8601).
end_time	Date-Time	No	End of the query window (ISO 8601).
movement	Array of String	No	Restrict results to one or more movements, using the explicit entry--exit syntax (e.g. N--W, N--N).
entry_zone_name	Array of String	No	Restrict results to one or more entry zones, by name.
exit_zone_name	Array of String	No	Restrict results to one or more exit zones, by name.
classification	Array of String	No	Restrict results to one or more classifications: PROSPECT, UNKNOWN, PERSON, BICYCLE, VEHICLE, LARGE_VEHICLE.
sub_classification	Array of String	No	Restrict results to one or more sub-classifications: none, pedestrian, car, truck, bicycle, bus, trailer, tram.
user_classification	Array of String	No	Restrict results to one or more user-defined classifications.
interval_seconds	Integer	No	Aggregation bucket width, in seconds. Defaults to the configured

Parameter	Type	Required	Description
			default_aggregation_interval_seconds and must satisfy the minimum and factor requirements set in the server configuration.
timezone	String	No	Overrides configured timezone for timestamps.

Sample Response

JSON


```
{
 "query_start_time": "2026-08-10T11:55:00Z",
 "query_end_time": "2026-08-10T12:00:00Z",
 "interval_seconds": 300,
 "total_buckets": 2,
 "timezone": "UTC",
 "units": "metric",
 "buckets": [
  {
   "bucket_start": "2026-08-10T11:55:00Z",
   "bucket_end": "2026-08-10T12:00:00Z",
   "entry_zone_name": "N",
   "entry_zone_id": 1,
   "exit_zone_name": "W",
   "exit_zone_id": 4,
   "movement": "N--W",
   "count": 18,
   "classification_counts": {
    "classification": { "VEHICLE": 16, "LARGE_VEHICLE": 2 },
    "sub_classification": { "car": 15, "truck": 2, "bus": 1 },
    "user_classification": {}
   }
  },
  {
   "bucket_start": "2026-08-10T11:55:00Z",
   "bucket_end": "2026-08-10T12:00:00Z",
   "entry_zone_name": "S",
   "entry_zone_id": 2,
```

```
      "exit_zone_name": "N",
      "exit_zone_id": 1,
      "movement": "S--N",
      "count": 24,
      "classification_counts": {
        "classification": { "VEHICLE": 22, "BICYCLE": 2 },
        "sub_classification": { "car": 22, "bicycle": 2 },
        "user_classification": { "delivery": 3 }
      }
    }
  ]
}
```


4.3.3 Zone Metadata Aggregation (zone_metadata)

Retrieves zone metadata (object detection) records aggregated into fixed-length time buckets, with per-bucket classification and speed-bin counts. Use this endpoint for dashboards and reports; use object_events (Section 4.3.1) when individual records are needed.

Detail	Value
Endpoint	https://<edge-box-address>/analytics/api/v1/zone_metadata
Method	GET

Parameters

Parameter	Type	Required	Description
start_time	Date-Time	No	Start of the query window (ISO 8601).
end_time	Date-Time	No	End of the query window (ISO 8601).
zone_name	String	No	Restrict results to a single zone, by name.
zone_id	Integer	No	Restrict results to a single zone, by ID.

Parameter	Type	Required	Description
classification	Array of String	No	Restrict results to one or more classifications.
sub_classification	Array of String	No	Restrict results to one or more sub-classifications.
interval_seconds	Integer	No	Aggregation bucket width, in seconds.
deduplicate_objects	Boolean	No	Overrides deduplicate_objects setting.
timezone	String	No	Overrides configured timezone for timestamps.
imperial_measuring_unit	Boolean	No	Overrides the configured unit system.

Sample Response

JSON


```
{
  "query_start_time": "2026-08-10T11:00:00Z",
  "query_end_time": "2026-08-10T12:00:00Z",
  "interval_seconds": 300,
  "total_buckets": 1,
  "timezone": "UTC",
  "units": "metric",
  "buckets": [
    {
      "bucket_start": "2026-08-10T11:55:00Z",
      "bucket_end": "2026-08-10T12:00:00Z",
      "zone_id": 14,
      "zone_name": "Zone-14",
      "count": 37,
      "avg_height": 1.6,
      "avg_length": 4.4,
      "avg_width": 1.8,
      "avg_speed": 24.1,
      "p50_speed": 23.5,
```

```
      "p85_speed": 28.9,
      "avg_dwell_time_ms": 3150.0,
      "avg_num_samples": 39.2,
      "classification_counts": {
        "classification": { "VEHICLE": 30, "PERSON": 5, "BICYCLE": 2 },
        "sub_classification": { "car": 28, "truck": 2, "pedestrian": 5, "bicycle": 2 },
        "user_classification": {}
      },
      "speed_bin_counts": { "Slow": 4, "Moderate": 25, "Fast": 8 }
    }
  ]
}
```


4.3.4 Zone Occupancy Ratio Aggregation (zone_occupancy)

Retrieves, for each requested zone, the time-weighted fraction of each aggregation bucket during which the zone was occupied by at least one object.

Detail	Value
Endpoint	https://<edge-box-address>/analytics/api/v1/zone_occupancy
Method	GET

Parameters

Parameter	Type	Required	Description
start_time	Date-Time	No	Start of the query window (ISO 8601). Defaults to 5 minutes before current time.
end_time	Date-Time	No	End of the query window (ISO 8601). Defaults to current time.
zone_name	String	No	Restrict results to a single zone, by name.
zone_id	Integer	No	Restrict results to a single zone, by ID.

Parameter	Type	Required	Description
interval_seconds	Integer	No	Aggregation bucket width, in seconds.
timezone	String	No	Overrides configured timezone for timestamps.
imperial_measuring_unit	Boolean	No	Overrides the configured unit system.

Sample Response

JSON


```
{
  "query_start_time": "2026-08-10T11:55:00Z",
  "query_end_time": "2026-08-10T12:00:00Z",
  "interval_seconds": 300,
  "total_buckets": 1,
  "timezone": "UTC",
  "units": "metric",
  "buckets": [
    {
      "bucket_start": "2026-08-10T11:55:00Z",
      "bucket_end": "2026-08-10T12:00:00Z",
      "zone_id": 14,
      "zone_name": "Zone-14",
      "avg_occupancy_rate": 0.42,
      "count": 37
    }
  ]
}
```


4.3.5 Zone Queue Length Aggregation (zone_queue_length)

Retrieves, for each requested zone, aggregated queue-length statistics per time bucket — average, median, 85th-percentile, and current queue length, plus average queued-object count.

Detail	Value
Endpoint	https://<edge-box-address>/analytics/api/v1/zone_queue_length

Detail	Value
Method	GET

Parameters

Parameter	Type	Required	Description
start_time	Date-Time	No	Start of the query window (ISO 8601).
end_time	Date-Time	No	End of the query window (ISO 8601).
zone_name	String	No	Restrict results to a single zone, by name.
zone_id	Integer	No	Restrict results to a single zone, by ID.
interval_seconds	Integer	No	Aggregation bucket width, in seconds.
timezone	String	No	Overrides configured timezone for timestamps.
imperial_measuring_unit	Boolean	No	Overrides the configured unit system.

Sample Response

JSON


```
{
  "query_start_time": "2026-08-10T11:55:00Z",
  "query_end_time": "2026-08-10T12:00:00Z",
  "interval_seconds": 300,
  "total_buckets": 1,
  "timezone": "UTC",
  "units": "metric",
  "buckets": [
    {
      "bucket_start": "2026-08-10T11:55:00Z",
      "bucket_end": "2026-08-10T12:00:00Z",
      "zone_id": 22,
      "zone_name": "Zone-22",
```

```
    "avg_current_queue_length": 12.4,
    "avg_queue_length": 11.8,
    "median_queue_length": 10.5,
    "p85_queue_length": 18.2,
    "avg_queue_object_count": 6.1,
    "count": 12
   }
  ]
}
```

5. Category 3 — Real-Time gRPC Streaming API

5.1 Overview

The Real-Time API provides live access to **frame data** (location, class type, and tracking ID of each object within a frame), **occupancy changes** for each phase (virtual inductive loops that are called upon the presence of vehicles in defined regions/loops), and **phase changes** at intersections (traffic light status). It can be used to build advanced applications on top of the BlueCity traffic monitoring solution.
The API uses the **gRPC** protocol over a long-lived, TLS-secured HTTP/2 connection. You can either use the BlueCity Python client module or build your own client based on the standards in this section.
Note on classification: although this category shares Authentication Scheme A with the Cloud-Based Analytics API, it is documented as a separate category because it differs in protocol (gRPC/protobuf vs REST/JSON), consumption model (persistent server stream vs request/response), and data nature (live events vs historical aggregates).

5.2 Authentication and Access

Two access modes are supported. The connection type parameter identifies the mode: **type 2** = JWT access, **type 3** = service-token access.

Mode	Credentials	Description
User (JWT)	Username + password on core.api.bluecity.ai	Obtain a JWT access token per Section 2.1.1
Service	Service name + service access token	For integrations that are provisioned as a service; the authentication step is skipped and the shared service token is used directly.

5.3 Communication Flow

The communication flow consists of three steps: **Authentication** (if applicable) → **Routing** → **Connecting**.

5.3.1 Routing (Beacon)

Real-time servers use a look-aside load-balancing architecture. Before connecting, clients must ask **Beacon** (the real-time server load balancer) for the correct real-time server endpoint and its SSL public key for the desired sensor UDID. Send an HTTP request (GET/POST) to a beacon address, e.g. https://realtime.us.beacon.1.api.bluecity.ai/route, with:

Field	Description
UDID	UDID of the desired sensor.
token	Your JWT access token or service access token.
type	Connection type: 2 for JWT access, 3 for service-token access.

Successful response (HTTP 200):

```json
{
  "address": "<realtime server address>",
  "key": "<public key>"
}
```

If your connection to a real-time server goes down for any reason, you must query the Beacon again for a fresh address and key before reconnecting.

5.3.2 Connecting (gRPC)

Connect to the returned address over a TLS channel using the returned public key. Communication happens via the **Subscriber** service:

```protobuf
service Subscriber {
   // this function should be called by clients.
   rpc subscribe(SubscriptionRequest) returns (stream HyperParameter) {}
   rpc ping(Empty) returns (Empty) {}
}
```

```
message SubscriptionRequest {
  bool initial = 1;
}
```

subscribe is a server-streaming RPC: the client sends a SubscriptionRequest (whose initial flag indicates whether initial values are requested) and receives a stream of HyperParameter messages. Authenticate the connection by placing the following in the request header/metadata:

Metadata Key	Description
bct-udid	UDID of the desired sensor.
token	Authentication token (JWT or service access token).
type	Connection type: 2 for JWT, 3 for service token.

With valid credentials, you hold a long-lived HTTP/2 connection to the server.

5.3.3 Reconnection and Queued Data

If you lose the connection, the server sends all data queued for your user immediately after reconnection. If your downtime exceeds the maximum length of the server-side queue, the server first sends all queued messages, then sends a message with *absolute* flags for both **OccupancyChange** and **PhaseChange** events to signal a re-initialization caused by the missing data.

5.4 Data Structures

5.4.1 HyperParameter

The real-time server sends data as **HyperParameter** messages. A HyperParameter can contain **frame**, **phase change**, and **occupancy change** information (one to three of these per message), plus a shared ISO 8601 timestamp. Bundling simultaneous events in one message guarantees the ordered, simultaneous delivery of concurrent events, which three separate streams could not.

Protobuf

```
message HyperParameter {
```

```
    string timestamp = 1;
    Frame frame = 2;
    PhaseChange phaseChange = 3;
    OccupancyChange occupancyChange = 4;
}
```

5.4.2 Frame and FrameObject

Each **Frame** contains a timestamp and a list of detected road users (**FrameObject**). If data comes from a virtual sensor (a UDID representing fused data from multiple sensors), the frame also carries lastTimesSeen, indicating the time of the last received frame from each participating sensor.

Protobuf

```
message Frame {
    string timestamp = 1;
    repeated FrameObject objects = 2;
    // Only available on fused data
    repeated LastTimeSeen lastTimesSeen = 3;
}

message LastTimeSeen {
    string udid = 1;
    double time = 2;
}
```

Each **FrameObject** represents one road user. Every object carries a unique ID that persists across all frames in which it appears; its location in metres relative to the sensor; the width, length, and rotation (radians) of its bounding box; and its class type. Optional attributes (speed, centerZ, height, accuracy) are not always guaranteed to be sent. For virtual (fused) sensors, confidences indicates the detection confidence in each participating sensor.

Protobuf

```
message FrameObject {
    string id = 1;
    float centerX = 2;
    float centerY = 3;
    float width = 4;
    float length = 5;
```

```
    float rotation = 6;
    string classType = 7;
    // Optional
    float speed = 8;
    float centerZ = 9;
    float height = 10;
    float accuracy = 11;
    // Only available on fused data
    repeated ConfidenceObeject confidences = 12;
}

message ConfidenceObeject {
    float confidence = 1;
    string udid = 2;
}
```

Real-Time Class Types

Code	Class Type
0	Pedestrian (old version API)
1	Vehicle (old version API)
2	Car
3	Van
4	Truck
5	Bus
6	SUV
7	Construction_vehicle
8	Trailer
9	Tractor
10	Pedestrian
11	Person_sitting
12	Cyclist
13	Bicycle

Code	Class Type
14	Person
15	Motorcycle
16	Rollerblader
17	Cyclist

5.4.3 PhaseChange

A **PhaseChange** event notifies clients of traffic-light changes across phases. It contains a timestamp, a list of changed phases, and an absolute flag indicating whether the message includes information for all phases.

```protobuf
message PhaseChange {
  string timestamp = 1;
  repeated Phase phases = 2;
  bool absolute = 3;
}

message Phase {
  string phaseNumber = 1;
  int32 status = 2;
  string timestamp = 3;
}
```

On first connection, the client receives a PhaseChange with absolute = true containing all phases at the intersection and their last status — useful for application initialization. When absolute = true, each inner Phase carries its own timestamp of the latest change; when absolute = false, Phase messages carry no timestamp. Phase status values: **0** Invalid, **1** Green, **2** Yellow, **3** Red.

5.4.4 OccupancyChange

Occupancy changes are sent as **OccupancyChange** messages, containing multiple occupancy records and an absolute flag similar to PhaseChange. Each **Occupancy** carries its own timestamp. Occupancies are associated with phases and identified by the label of the phase in which they occur. status = true means the virtual loop is occupied; false means unoccupied.

```protobuf
message OccupancyChange {
  repeated Occupancy occupancies = 1;
  bool absolute = 2;
}

message Occupancy {
  string phaseLabel = 1;
  bool status = 2;
  string timestamp = 3;
}
```

5.5 Python Client Module (BCT Realtime Subscriber SDK)

5.5.1 Installation

Install the subscriber package from the provided wheel file:

```shell
python3 -m pip install bctRealtimeSubscriber-x.x.x-py3-none-any.whl
```

Requirements for the current SDK sample: Python >= 3.8 and < 3.12. Projects using uv can declare the local wheel (e.g. bctRealtimeSDK_Subscriber-3.1.0-py3-none-any.whl) in pyproject.toml and run uv sync.
Two client variants exist, differing only in the authentication process; import BCTWSConnection from the module matching the repository shared with you. Keep credentials out of source control.

5.5.2 Realtime_subscriber_api (username/password)

```python
from realtime_subscriber.Realtime_subscriber_api import BCTWSConnection
```

```
stream = BCTWSConnection(
    "<UDID>",
    username="<username>",
    password="<password>",
    singleton=False,
    subscriptions=[
        BCTWSConnection.subscriptionOption.LOOP_CHANGE,
        BCTWSConnection.subscriptionOption.PHASE_CHANGE,
        BCTWSConnection.subscriptionOption.FRAME,
    ],
)
```

The module communicates with the authentication service and fetches a JWT in the background. Depending on your access level, you can subscribe to **Frame**, **PhaseChange**, **OccupancyChange**, and/or **DetectionFrame** events.

5.5.3 Realtime_subscriber_service (service token)

Python

```
from realtime_subscriber.Realtime_subscriber_service import BCTWSConnection

token = "<Token>"
stream = BCTWSConnection(
    "<UDID>",
    service="<service>",
    singleton=False,
    subscriptions=[BCTWSConnection.subscriptionOption.FRAME],
    token=token,
)
```

5.5.4 Consuming Data

With singleton=True (default), you receive whole **HyperParameter** messages as the server sends them:

Python

```
while True:
    data = stream.get_hyperparameter()
```

```
    print(data.frame)
    print(data.occupancyChange)
    print(data.phaseChange)
```

With singleton=False, data is delivered separately per event type; you can use multithreading to process each event stream simultaneously via stream.get_frame(), stream.get_occupancy(), and stream.get_phase().
The connection is a persistent, long-lived HTTP/2 connection: create the connection object once and call the get_...() functions to consume data queued client-side, in the order sent. Received data is stored in background FIFO queues whose size can be set at construction.

5.5.5 Realtime_subscriber_archive (offline testing)

For testing, you may be given an archive file containing real-time data for a specific time range:

```python
from realtime_subscriber.Realtime_subscriber_archive import ArchiveReader

reader = ArchiveReader("<filename>.archive", fps=10)
while True:
    hyperParam = reader.get_hyperparameter()
    print(hyperParam.frame)
    print(hyperParam.occupancyChange)
    print(hyperParam.phaseChange)
```

5.6 Building a Custom Client

To build a client in another language, use the shared .proto file to generate the data structures, then implement the three-step communication flow of Section 5.3: obtain a token (Section 2.1.1, skipped for service access), query the Beacon for the server address and public key, and open a TLS gRPC channel to the **Subscriber** service with bct-udid, token, and type in the call metadata.

6. Appendices

Appendix A — Swagger and Reference Links

Resource	URL
Core APIs Swagger	https://core.api.bluecity.ai/schema/swagger-ui/
Safety APIs Swagger	https://safety.api.bluecity.ai/api/schema/swagger-ui/
SPM APIs Swagger	https://spm.api.bluecity.ai/api/schema/swagger-ui/
Edge Zone Analytics Swagger	https://<edge-box-address>/analytics/api/v1/swagger/ui/ (on the edge box's local network)
Token endpoint	https://core.api.bluecity.ai/api/token/
Token refresh endpoint	https://core.api.bluecity.ai/api/token/refresh/
Beacon (real-time routing)	https://realtime.us.beacon.1.api.bluecity.ai/route

Appendix B — Glossary

Term	Definition
Edge box	The on-site BlueCity sensing/compute device installed at an intersection.
UDID	Unique identifier of an edge box / sensor, or of a virtual (fused) sensor.
TMC	Turning Movement Counts — aggregated counts of movements through an intersection.
PET	Post Encroachment Time — a surrogate

Term	Definition
	safety measure of the time gap between two road users passing through the same conflict point.
SPM	Signal Performance Measures — analytics describing traffic signal operation quality (occupancy ratio, arrivals, green allocation, delay, etc.).
Phase	A traffic signal phase (movement group) at the intersection, identified by a phase ID/number.
Virtual loop	A software-defined detection zone emulating an inductive loop, used for occupancy detection.
Beacon	The real-time service's look-aside load balancer, which routes clients to the correct streaming server.
HyperParameter	The top-level protobuf message of the real-time stream, bundling frame, phase change, and occupancy change events.

Appendix C — Support

For additional support, access requests (core.api.bluecity.ai accounts, service tokens, SDK wheel files, proto files), please contact the Ouster BlueCity support team.